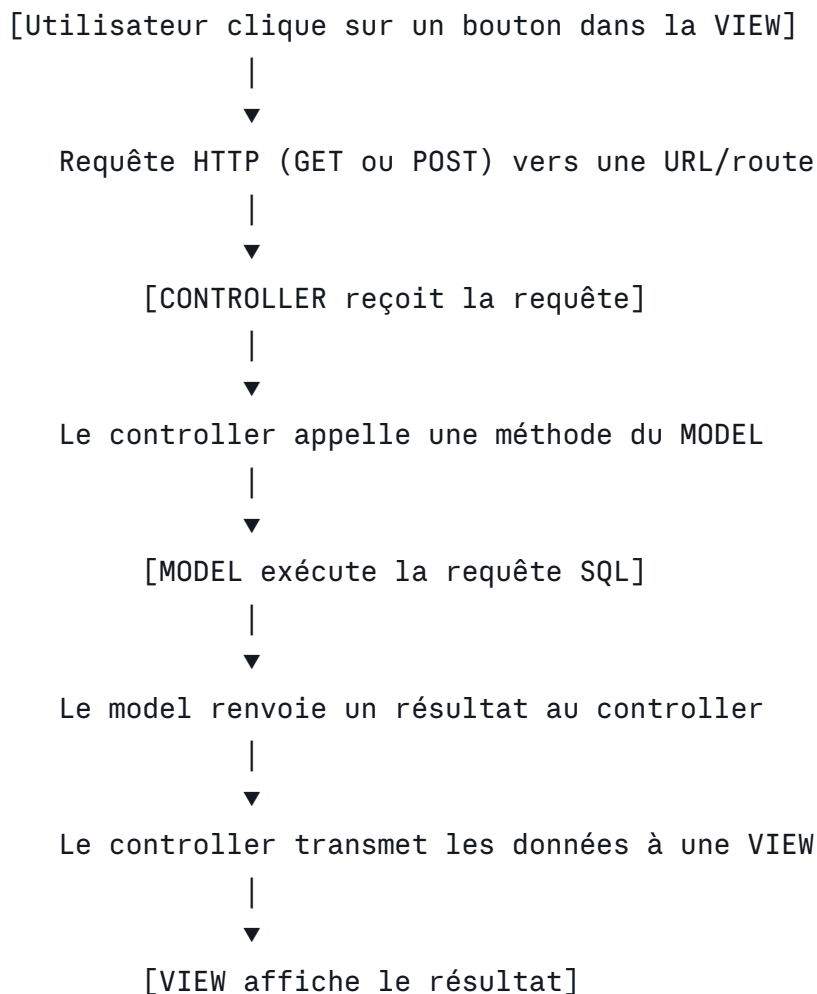


CodeIgniter — Exécuter une requête SQL au clic d'un bouton

Ce document explique, étape par étape, comment déclencher une requête SQL depuis une **vue** (clic sur un bouton), la faire transiter par un **controller**, et l'exécuter dans un **model**, selon l'architecture MVC de CodeIgniter (CI3/CI4 — la syntaxe CI4 est utilisée ici, avec les variantes CI3 signalées).

Le flux est générique : il fonctionne pour **n'importe quelle requête** (SELECT, INSERT, UPDATE, DELETE), il suffit d'adapter le contenu du model.

1. Vue d'ensemble du flux



Deux façons courantes de déclencher l'action :

1. **Formulaire HTML classique** (le bouton soumet un formulaire → rechargement de page).
2. **AJAX (fetch/jQuery)** (le bouton envoie une requête sans recharger la page).

On détaille les deux, en commençant par la méthode classique.

2. Prérequis : configuration de la base de données

Dans `app/Config/Database.php` (CI4) ou `application/config/database.php` (CI3), vérifiez que la connexion est bien configurée :

```
php

public array $default = [
    'DSN'          => '',
    'hostname'     => 'localhost',
    'username'     => 'root',
    'password'     => '',
    'database'     => 'ma_base',
    'DBDriver'     => 'MySQLi',
    ...
];
```

3. Méthode 1 — Bouton dans un formulaire (rechargement de page)

3.1 La route

Dans `app/Config/Routes.php` :

```
php

$routes->get('produits', 'ProduitController::index');
$routes->post('produits/executer', 'ProduitController::executerRequete');
```

En CI3, on utilise `application/config/routes.php` avec une syntaxe similaire :

```
$route['produits/executer'] = 'produit/executerRequete';
```

3.2 La View (formulaire avec bouton)

`app/Views/produits/index.php` :

php

```
<!DOCTYPE html>
<html lang="fr">
<head>
    <meta charset="UTF-8">
    <title>Liste des produits</title>
</head>
<body>

    <h1>Produits</h1>

    <!-- Le bouton se trouve dans un formulaire -->
    <form action="<?= site_url('produits/executer') ?>" method="post">
        <?= csrf_field() ?> <!-- protection CSRF, recommandé -->
        <button type="submit">Charger les produits</button>
    </form>

    <!-- Affichage du résultat si présent -->
    <?php if (isset($produits)) : ?>
        <table border="1">
            <tr>
                <th>ID</th>
                <th>Nom</th>
                <th>Prix</th>
            </tr>
            <?php foreach ($produits as $p) : ?>
                <tr>
                    <td><?= esc($p['id']) ?></td>
                    <td><?= esc($p['nom']) ?></td>
                    <td><?= esc($p['prix']) ?></td>
                </tr>
            <?php endforeach; ?>
        </table>
    <?php endif; ?>

</body>
</html>
```

Points clés :

- `esc()` échappe les données affichées (protection XSS).
- `csrf_field()` insère un champ caché anti-CSRF (CI4 l'exige par défaut si la protection CSRF est activée dans `Config/Filters.php` ou `Config/Security.php`).

3.3 Le Controller

`app/Controllers/ProduitController.php` :

php

```
<?php
```

```
namespace App\Controllers;
```

```
use App\Models\ProduitModel;
```

```
class ProduitController extends BaseController
```

```
{
```

```
    protected ProduitModel $produitModel;
```

```
    public function __construct()
```

```
    {
```

```
        // Instanciation du model
```

```
        $this->produitModel = new ProduitModel();
```

```
    }
```

```
    // Affiche simplement le formulaire, sans résultat
```

```
    public function index()
```

```
    {
```

```
        return view('produits/index');
```

```
    }
```

```
    // Appelée quand on clique sur le bouton (soumission du formulaire)
```

```
    public function executerRequete()
```

```
    {
```

```
        // Appel de la méthode du model qui exécute la requête SQL
```

```
        $produits = $this->produitModel->getTousLesProduits();
```

```
        // On transmet le résultat à la vue
```

```
        return view('produits/index', ['produits' => $produits]);
```

```
    }
```

```
}
```

En CI3, la syntaxe est comparable mais sans les types stricts ((protected
\$produitModel;)) et le chargement du model se fait via (\$this->load-
>model('Produit_model');).

3.4 Le Model

app/Models/ProduitModel.php):

php

```
<?php
```

```
namespace App\Models;
```

```
use CodeIgniter\Model;
```

```
class ProduitModel extends Model
```

```
{
```

```
    protected $table = 'produits';
```

```
    // Exemple avec le Query Builder (recommandé, sécurisé par défaut)
```

```
    public function getTousLesProduits(): array
```

```
    {
```

```
        return $this->db->table('produits')
            ->select('id, nom, prix')
            ->get()
            ->getResultArray();
    }
```

```
    // Exemple avec une requête SQL brute (si besoin de SQL personnalisé)
```

```
    public function getProduitsSql(): array
```

```
    {
```

```
        $query = $this->db->query('SELECT id, nom, prix FROM produits');
        return $query->getResultArray();
    }
```

```
    // Exemple avec un paramètre (requête préparée, échappement automatique)
```

```
    public function getProduitParId(int $id): ?array
```

```
    {
```

```
        $query = $this->db->query(
            'SELECT id, nom, prix FROM produits WHERE id = ?',
            [$id]
        );
        return $query->getRowArray();
    }
```

```
    // Exemple INSERT
```

```
    public function ajouterProduit(string $nom, float $prix): bool
```

```
    {
```

```
        return $this->db->table('produits')->insert([
            'nom' => $nom,
            'prix' => $prix,
        ]);
    }
```

```
    // Exemple UPDATE
```

```
    public function modifierProduit(int $id, array $donnees): bool
```

```

    }

    return $this->db->table('produits')
        ->where('id', $id)
        ->update($donnees);

    }

    // Exemple DELETE
    public function supprimerProduit(int $id): bool
    {
        return $this->db->table('produits')
            ->where('id', $id)
            ->delete();

    }
}

```

Astuce généralisation : n'importe quelle requête suit ce même schéma — une méthode du model qui utilise soit le Query Builder (`$this->db->table(...)`), soit `$this->db->query('SQL', [params])` pour du SQL brut avec liaison de paramètres (toujours utiliser des `?` ou des paramètres nommés, **jamais** de concaténation directe de variables dans la chaîne SQL, pour éviter les injections SQL).

4. Méthode 2 — Bouton en AJAX (sans rechargement de page)

Utile quand on veut afficher le résultat dynamiquement (ex. tableau qui se met à jour sans recharger la page).

4.1 La route (retourne du JSON)

```

php

$routes->post('produits/ajax-executer', 'ProduitController::ajaxExecuter');

```

4.2 La View

```

php

<button id="btnCharger">Charger les produits (AJAX)</button>

<div id="resultat"></div>

<script>
document.getElementById('btnCharger').addEventListener('click', function () {
    fetch('<?= site_url('produits/ajax-executer') ?>', {
        method: 'POST',
        headers: {
            'X-Requested-With': 'XMLHttpRequest',
            'Content-Type': 'application/json'
        }
    })
    .then(response => response.json())
    .then(data => {
        let html = '<table border="1"><tr><th>ID</th><th>Nom</th><th>Prix</th><tr>';
        data.forEach(p => {
            html += `<tr><td>${p.id}</td><td>${p.nom}</td><td>${p.prix}</td></tr>`;
        });
        html += '</table>';
        document.getElementById('resultat').innerHTML = html;
    })
    .catch(err => console.error(err));
});
</script>

```

4.3 Le Controller (renvoi du JSON)

```

php

public function ajaxExecuter()
{
    $produits = $this->produitModel->getTousLesProduits();

    // CodeIgniter fournit une méthode pratique pour répondre en JSON
    return $this->response->setJSON($produits);
}

```

4.4 Le Model

Identique à la section 3.4 — **le model ne change jamais** selon que l'appel vienne d'un formulaire classique ou d'AJAX ; seule la manière dont le controller renvoie la réponse diffère (`view()` vs `setJSON()`).

5. Récapitulatif du rôle de chaque composant

Composant	Rôle
View	Affiche le bouton/formulaire, envoie la requête HTTP, affiche le résultat
Route	Fait le lien entre l'URL appelée et la méthode du controller
Controller	Reçoit la requête, appelle le model, choisit comment répondre (vue ou JSON)
Model	Contient la logique d'accès aux données et exécute la requête SQL

6. Bonnes pratiques

- **Ne jamais** écrire de requête SQL directement dans le controller ou la view : toute la logique SQL reste dans le model.
- **Toujours** utiliser des requêtes préparées (`?` ou paramètres nommés) ou le Query Builder pour éviter les injections SQL.
- **Échapper** les données affichées dans les vues avec `esc()`.
- **Activer la protection CSRF** pour les formulaires qui modifient des données (POST/PUT/DELETE).
- Pour les actions qui modifient la base (INSERT/UPDATE/DELETE), utiliser la méthode HTTP **POST** (jamais GET), et valider les données côté controller avant de les transmettre au model.

7. Adapter ce schéma à n'importe quelle requête

Pour n'importe quelle nouvelle requête SQL :

1. Ajouter une méthode dans le **Model** correspondant à l'action désirée (SELECT/INSERT/UPDATE/DELETE), avec Query Builder ou `$this->db->query()`.
2. Ajouter une méthode dans le **Controller** qui appelle cette méthode du model.
3. Ajouter une **route** qui pointe vers cette méthode du controller.
4. Ajouter un **bouton** (formulaire ou AJAX) dans la **View** qui déclenche cette route.

Ce schéma en 4 étapes est réutilisable pour toute fonctionnalité basée sur une requête SQL déclenchée par une action utilisateur.